

codigo gerado pelo claudelA

```
#
=====

=====
# SISTEMA DE ANÁLISE DE ABASTECIMENTO DE ÁGUA
# Compatível com Google Colab e Python 3.x
# Bibliotecas: apenas pandas, matplotlib e tabulate (disponíveis no Colab)
#
=====
=====

import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import pandas as pd
from tabulate import tabulate

# -----
# CONFIGURAÇÕES VISUAIS
# -----

plt.rcParams.update({
    'figure.facecolor': '#FAFAFA',
    'axes.facecolor': '#FAFAFA',
    'axes.grid':      True,
    'grid.alpha':     0.4,
    'grid.linestyle': '--',
    'font.family':    'DejaVu Sans',
    'axes.spines.top': False,
    'axes.spines.right': False,
})

CORES = {
    'demanda': '#378ADD',
    'entregue': '#1D9E75',
    'perdido':  '#E24B4A',
    'custo':    '#BA7517',
    'baixo':    '#27500A',
    'medio':    '#633806',
    'alto':     '#A32D2D',
    'baixo_bg': '#C0DD97',
    'medio_bg': '#FAC775',
    'alto_bg':  '#F09595',
}

# -----
# FUNÇÕES DE ENTRADA
# -----
```

```

def ler_float(mensagem, minimo=0.0, maximo=None):
    """Lê um número float com validação."""
    while True:
        try:
            valor = float(input(mensagem))
            if valor < minimo:
                print(f" ⚠ Valor deve ser ≥ {minimo}. Tente novamente.")
                continue
            if maximo is not None and valor > maximo:
                print(f" ⚠ Valor deve ser ≤ {maximo}. Tente novamente.")
                continue
            return valor
        except ValueError:
            print(" ⚠ Entrada inválida. Digite um número.")

def ler_int(mensagem, minimo=1, maximo=None):
    """Lê um número inteiro com validação."""
    while True:
        try:
            valor = int(input(mensagem))
            if valor < minimo:
                print(f" ⚠ Valor deve ser ≥ {minimo}. Tente novamente.")
                continue
            if maximo is not None and valor > maximo:
                print(f" ⚠ Valor deve ser ≤ {maximo}. Tente novamente.")
                continue
            return valor
        except ValueError:
            print(" ⚠ Entrada inválida. Digite um número inteiro.")

def coletar_dados():
    """Solicita os dados de todos os bairros ao usuário."""
    print("\n" + "="*60)
    print(" SISTEMA DE ANÁLISE DE ABASTECIMENTO DE ÁGUA")
    print("="*60)

    n = ler_int("\nQuantos bairros serão analisados? ", minimo=1, maximo=20)
    bairros = []

    for i in range(n):
        print(f"\n{'-'*40}")
        print(f" Bairro {i+1} de {n}")
        print(f"{'-'*40}")

        nome = input(" Nome do bairro: ").strip() or f"Bairro {i+1}"
        demanda = ler_float(" Demanda diária estimada (m³): ", minimo=0.01)
        capac = ler_float(" Capacidade do reservatório (m³): ", minimo=0)

```

```
vinicial = ler_float(" Volume inicial disponível (m³): ", minimo=0)
perda    = ler_float(" Percentual de perda na tubulação (%): ", minimo=0, maximo=100)
custo    = ler_float(" Custo de bombeamento por m³ (R$): ", minimo=0)
```

```
bairros.append({
    'nome': nome,
    'demanda': demanda,
    'capac': capac,
    'vinicial': vinicial,
    'perda': perda,
    'custo': custo,
})
```

```
return bairros
```

```
# -----
# PROCESSAMENTO
# -----
```

```
def classificar_risco(autonomia):
    if autonomia > 3:
        return 'Baixo'
    elif autonomia >= 1:
        return 'Médio'
    else:
        return 'Alto'
```

```
def calcular_indicadores(bairros):
    """Calcula todos os indicadores para cada bairro."""
    resultados = []
    for b in bairros:
        vol_perdido = round(b['demanda'] * b['perda'] / 100, 2)
        vol_entregue = round(b['demanda'] - vol_perdido, 2)
        custo_diario = round(b['demanda'] * b['custo'], 2)
        autonomia = round(b['vinicial'] / b['demanda'], 2) if b['demanda'] > 0 else 0
        risco = classificar_risco(autonomia)
```

```
    resultados.append({
        **b,
        'vol_perdido': vol_perdido,
        'vol_entregue': vol_entregue,
        'custo_diario': custo_diario,
        'autonomia': autonomia,
        'risco': risco,
    })
    return resultados
```

```
# -----
```

SAÍDA — TABELA

```
def exibir_tabela(resultados):
```

```
    """Exibe tabela formatada no terminal."""
```

```
    print("\n" + "="*60)
```

```
    print("  RESULTADOS POR BAIRRO")
```

```
    print("="*60)
```

```
    linhas = []
```

```
    for r in resultados:
```

```
        emoji_risco = {'Baixo': '🟢', 'Médio': '🟡', 'Alto': '🔴'}.get(r['risco'], "")
```

```
        linhas.append([
```

```
            r['nome'],
```

```
            f'{r["demanda"]:.1f}',
```

```
            f'{r["vol_entregue"]:.1f}',
```

```
            f'{r["vol_perdido"]:.1f}',
```

```
            f'R$ {r["custo_diario"]:.2f}',
```

```
            f'{r["autonomia"]:.1f}',
```

```
            f'{emoji_risco} {r["risco"]}',
```

```
        ])
```

```
    headers = ['Bairro', 'Demanda\n(m³/dia)', 'Entregue\n(m³/dia)',
```

```
               'Perdido\n(m³/dia)', 'Custo\ndiário', 'Autonomia\n(dias)', 'Risco']
```

```
    print(tabulate(linhas, headers=headers, tablefmt='rounded_outline', stralign='center',
numalign='right'))
```

```
def exibir_resumo(resultados):
```

```
    """Exibe métricas consolidadas e bairro mais crítico."""
```

```
    total_demanda = sum(r['demanda'] for r in resultados)
```

```
    total_entregue = sum(r['vol_entregue'] for r in resultados)
```

```
    total_custo = sum(r['custo_diario'] for r in resultados)
```

```
    eficiencia = (total_entregue / total_demanda * 100) if total_demanda > 0 else 0
```

```
    critico = min(resultados, key=lambda r: r['autonomia'])
```

```
    print("\n" + "="*60)
```

```
    print("  RESUMO DO SISTEMA")
```

```
    print("="*60)
```

```
    print(f" Demanda total diária : {total_demanda:.1f} m³")
```

```
    print(f" Volume total entregue : {total_entregue:.1f} m³")
```

```
    print(f" Eficiência geral : {eficiencia:.1f}%")
```

```
    print(f" Custo total diário : R$ {total_custo:.2f}")
```

```
    n_alto = sum(1 for r in resultados if r['risco'] == 'Alto')
```

```
    n_medio = sum(1 for r in resultados if r['risco'] == 'Médio')
```

```
    n_baixo = sum(1 for r in resultados if r['risco'] == 'Baixo')
```

```
    print(f" Distribuição de risco : 🟢 Baixo={n_baixo} 🟡 Médio={n_medio} 🔴
```

```
Alto={n_alto}")
```

```

print("\n" + "-"*60)
print(" ⚠ BAIRRO MAIS CRÍTICO")
print("-"*60)
print(f" Nome      : {critico['nome']}")
print(f" Autonomia  : {critico['autonomia']:.1f} dia(s)")
print(f" Risco      : {critico['risco']}")
print(f" Entregue   : {critico['vol_entregue']:.1f} m³/dia")
print(f" Custo      : R$ {critico['custo_diario']:.2f}/dia")
print("-"*60)

```

```

# -----
# GRÁFICOS
# -----

```

```

def cor_risco(risco):
    mapa = {'Baixo': CORES['baixo_bg'], 'Médio': CORES['medio_bg'], 'Alto':
CORES['alto_bg']}
    return mapa.get(risco, '#CCCCCC')

```

```

def grafico_demanda_entregue(ax, resultados):
    """Gráfico de barras: demanda × volume entregue."""
    nomes = [r['nome'] for r in resultados]
    demanda = [r['demanda'] for r in resultados]
    entregue = [r['vol_entregue'] for r in resultados]
    x = range(len(nomes))
    largura = 0.38

    ax.bar([i - largura/2 for i in x], demanda, width=largura,
           color=CORES['demanda'], label='Demanda', alpha=0.88, zorder=3)
    ax.bar([i + largura/2 for i in x], entregue, width=largura,
           color=CORES['entregue'], label='Vol. entregue', alpha=0.88, zorder=3)

    for i, (d, e) in enumerate(zip(demanda, entregue)):
        ax.text(i - largura/2, d + max(demanda)*0.01, f'{d:,.0f}',
                ha='center', va='bottom', fontsize=8, color=CORES['demanda'], fontweight='bold')
        ax.text(i + largura/2, e + max(demanda)*0.01, f'{e:,.0f}',
                ha='center', va='bottom', fontsize=8, color=CORES['entregue'], fontweight='bold')

    ax.set_xticks(list(x))
    ax.set_xticklabels(nomes, fontsize=9, rotation=15 if len(nomes) > 5 else 0, ha='right')
    ax.set_ylabel('Volume (m³/dia)', fontsize=9)
    ax.set_title('Demanda × Volume Entregue', fontsize=11, fontweight='bold', pad=10)
    ax.legend(fontsize=9)
    ax.set_ylim(0, max(demanda) * 1.18)

```

```

def grafico_custo(ax, resultados):
    """Gráfico de barras: custo diário por bairro."""

```

```

nomes = [r['nome'] for r in resultados]
custos = [r['custo_diario'] for r in resultados]
cores = [cor_risco(r['risco']) for r in resultados]
x = range(len(nomes))

barras = ax.bar(x, custos, color=cores, edgecolor='white', linewidth=0.8, zorder=3,
alpha=0.92)

for bar, val in zip(barras, custos):
    ax.text(bar.get_x() + bar.get_width()/2, val + max(custos)*0.01,
            f'R$\n{val:,.0f}', ha='center', va='bottom', fontsize=8, fontweight='bold')

ax.set_xticks(list(x))
ax.set_xticklabels(nomes, fontsize=9, rotation=15 if len(nomes) > 5 else 0, ha='right')
ax.set_ylabel('Custo (R$)', fontsize=9)
ax.set_title('Custo Diário de Bombeamento', fontsize=11, fontweight='bold', pad=10)
ax.set_ylim(0, max(custos) * 1.22)

legendas = [
    mpatches.Patch(color=CORES['baixo_bg'], label='Risco Baixo'),
    mpatches.Patch(color=CORES['medio_bg'], label='Risco Médio'),
    mpatches.Patch(color=CORES['alto_bg'], label='Risco Alto'),
]
ax.legend(handles=legendas, fontsize=8, loc='upper right')

def grafico_risco(ax, resultados):
    """Gráfico de pizza: distribuição de risco."""
    contagem = {'Baixo': 0, 'Médio': 0, 'Alto': 0}
    for r in resultados:
        contagem[r['risco']] += 1

    labels = [k for k, v in contagem.items() if v > 0]
    valores = [v for v in contagem.values() if v > 0]
    cores = [{'Baixo': CORES['baixo_bg'], 'Médio': CORES['medio_bg'], 'Alto':
CORES['alto_bg']}][l] for l in labels]
    explode = [0.04] * len(labels)

    wedges, texts, autotexts = ax.pie(
        valores, labels=labels, colors=cores, explode=explode,
        autopct=lambda p: f'{p:.1f}%\n({int(round(p*sum(valores)/100)}) bairro{"s" if
int(round(p*sum(valores)/100)}>1 else ""})',
        startangle=90, pctdistance=0.72, textprops={'fontsize': 9},
        wedgeprops={'edgecolor': 'white', 'linewidth': 1.5}
    )
    for at in autotexts:
        at.set_fontsize(8)
        at.set_fontweight('bold')

```

```

ax.set_title('Distribuição dos Níveis de Risco', fontsize=11, fontweight='bold', pad=10)

def grafico_autonomia(ax, resultados):
    """Gráfico de barras horizontais: autonomia por bairro."""
    nomes = [r['nome'] for r in resultados]
    autonomia = [r['autonomia'] for r in resultados]
    cores = [cor_risco(r['risco']) for r in resultados]
    y = range(len(nomes))

    barras = ax.barh(list(y), autonomia, color=cores, edgecolor='white',
                     linewidth=0.8, zorder=3, alpha=0.92, height=0.55)

    ax.axvline(x=1, color=CORES['alto'], linestyle='--', linewidth=1.2, alpha=0.7, label='Limiar
1 dia (Alto/Médio)')
    ax.axvline(x=3, color=CORES['medio'], linestyle='--', linewidth=1.2, alpha=0.7,
label='Limiar 3 dias (Médio/Baixo)')

    for bar, val in zip(barras, autonomia):
        ax.text(val + max(autonomia)*0.01, bar.get_y() + bar.get_height()/2,
                f'{val:.1f}d', va='center', fontsize=8, fontweight='bold')

    ax.set_yticks(list(y))
    ax.set_yticklabels(nomes, fontsize=9)
    ax.set_xlabel('Autonomia (dias)', fontsize=9)
    ax.set_title('Autonomia dos Reservatórios', fontsize=11, fontweight='bold', pad=10)
    ax.set_xlim(0, max(autonomia) * 1.22)
    ax.legend(fontsize=8, loc='lower right')

def gerar_graficos(resultados):
    """Gera painel com 4 gráficos."""
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle('Sistema de Abastecimento de Água — Análise Comparativa',
                fontsize=14, fontweight='bold', y=1.01)
    fig.patch.set_facecolor('#F4F4F4')

    grafico_demanda_entregue(axes[0, 0], resultados)
    grafico_custo(axes[0, 1], resultados)
    grafico_risco(axes[1, 0], resultados)
    grafico_autonomia(axes[1, 1], resultados)

    plt.tight_layout(pad=2.5)
    plt.savefig('abastecimento_agua.png', dpi=150, bbox_inches='tight',
                facecolor='#F4F4F4', edgecolor='none')
    plt.show()
    print("\n  Gráfico salvo como 'abastecimento_agua.png'")

# -----
# EXPORTAR TABELA CSV

```

```

# -----

def exportar_csv(resultados):
    df = pd.DataFrame([
        'Bairro':          r['nome'],
        'Demanda (m3/dia)': r['demanda'],
        'Capacidade (m3)':  r['capac'],
        'Volume Inicial (m3)': r['vinicial'],
        'Perda (%)':        r['perda'],
        'Custo/m3 (R$)':     r['custo'],
        'Vol. Entregue (m3)': r['vol_entregue'],
        'Vol. Perdido (m3)': r['vol_perdido'],
        'Custo Diario (R$)': r['custo_diario'],
        'Autonomia (dias)':  r['autonomia'],
        'Risco':             r['risco'],
    ] for r in resultados])
    df.to_csv('abastecimento_agua.csv', index=False, encoding='utf-8-sig')
    print(" 📄 Resultados exportados para 'abastecimento_agua.csv'")

# -----
# PROGRAMA PRINCIPAL
# -----

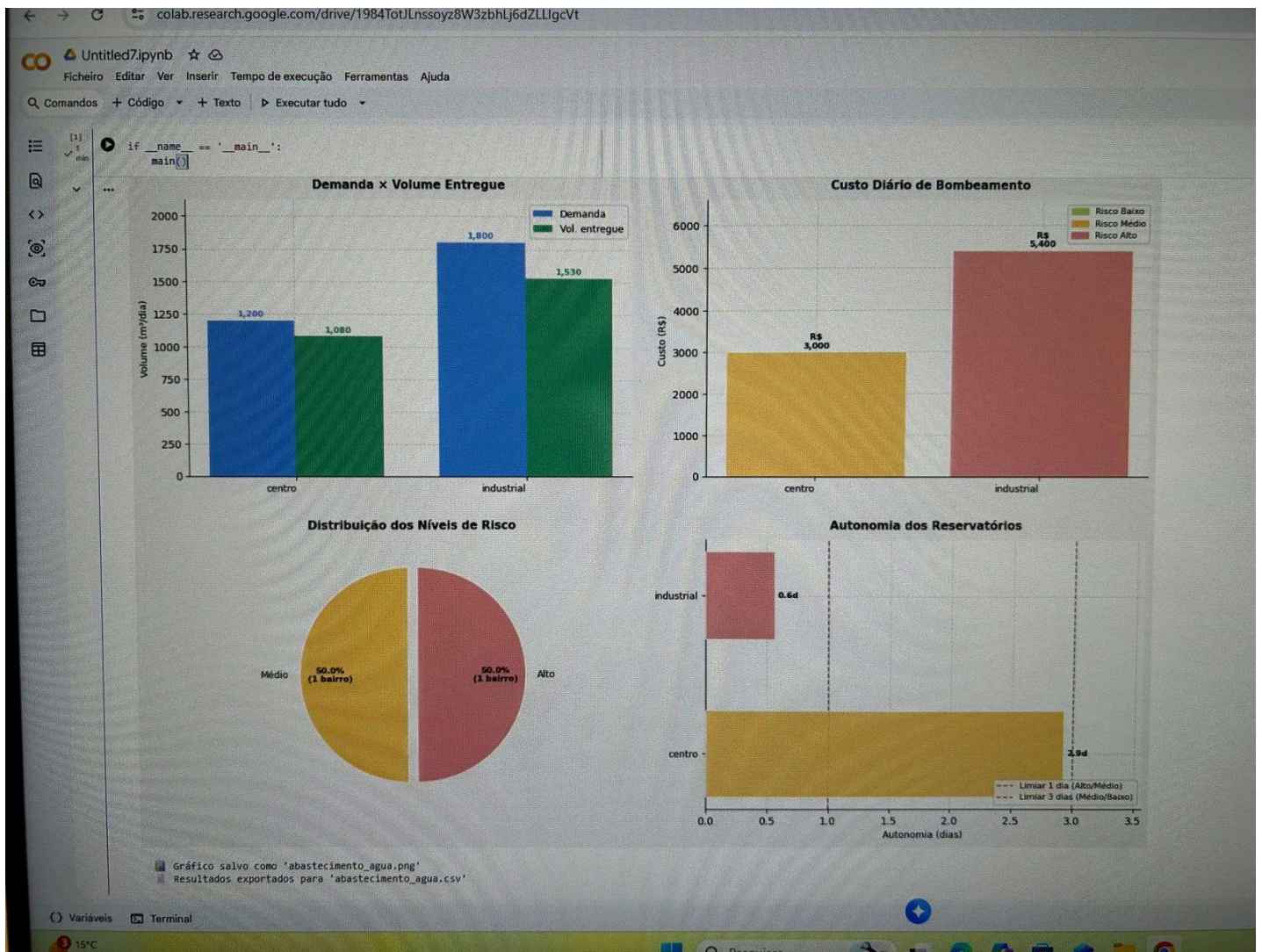
def main():
    bairros = coletar_dados()
    resultados = calcular_indicadores(bairros)

    exibir_tabela(resultados)
    exibir_resumo(resultados)
    gerar_graficos(resultados)
    exportar_csv(resultados)

    print("\n ✅ Análise concluída.\n")

if __name__ == '__main__':
    main()

```

Capacidade do sistema : 10000 m³
 Volume inicial disponível (m³): 1000
 Percentual de perda na tubulação (%): 15
 Custo de bombeamento por m³ (R\$): 3

RESULTADOS POR BAIRRO

Bairro (m ³ /dia)	Demanda Entregue (m ³ /dia)	Perdido (m ³ /dia)	Custo diário (dias)	Risco	Autonomia	
centro	1,200.0	1,080.0	120	R\$ 3,000.00	2.9	● Médio
industrial	1,800.0	1,530.0	270	R\$ 5,400.00	0.6	● Alto

RESUMO DO SISTEMA

Demanda total diária : 3,000.0 m³
 Volume total entregue : 2,610.0 m³
 Eficiência geral : 87.0%
 Custo total diário : R\$ 8,400.00
 Distribuição de risco : ● Baixo=0 ● Médio=1 ● Alto=1

⚠ BAIRRO MAIS CRÍTICO

Nome : industrial
 Autonomia : 0.6 dia(s)
 Risco : Alto
 Entregue : 1,530.0 m³/dia
 Custo : R\$ 5,400.00/dia

Sistema de Abastecim

variáveis  Terminal

15°C
 Ensolarado

Vision